A focused review of Topchyi's proposed P = NP proof

Technical note for `tarstars/pnp` | 15 October 2026

Prepared with ChatGPT

Abstract

The argument for Theorem 5.1 in Dmytro Topchyi's manuscript of 11 September 2026 does not establish P = NP as written. We give an exact counterexample to the power-mean identity used in Corollary 5.4 and distinguish three further proof obligations: deriving SAT labels from the input, recovering exact answers from approximate fields, and proving a uniform polynomial bit-complexity bound. This is a focused review of the computational argument, not a verdict on every definition of plafales, and it implies no separation of P and NP.

1 Scope and the computational claim

Theorem 5.1 claims a polynomial-time algorithm for Boolean satisfiability [1, p. 10]. For the constructive route proposed, the relevant target is a single deterministic algorithm A and a fixed polynomial p such that, for every encoded formula S of length n,

$$A(S) = \mathrm{SAT}(S) \in \{0, 1\}, \qquad T_A(S) \le p(n).$$

This is the standard input-length, worst-case notion of polynomial time [2, pp. 1–2, 4–5]. The issue is not the absence of program code, but whether the mathematical construction establishes correctness and this bound. We examine Section 5 through its deterministic Turing-machine (DTM) implementation and comments (pp. 10–17); no assessment of all earlier definitions or of the cited approximation and kinetic theories is needed for the objections below.

2 Encoding an answer is not deriving it

The manuscript assigns decreasing positive levels g_i and prescribes the terminal value g_i for a satisfiable instance S_i, or g_{i+1} otherwise [1, p. 11, displayed system and following sentence]. With $b_i = \mathrm{SAT}(S_i)$ this is simply

$$\lambda_i = g_{i+1} + (g_i - g_{i+1})b_i. \tag{1}$$

Thus a sufficiently accurate value of λ_i encodes the desired answer. The field $f(x, y, t) = \sum_i \lambda_i(t)L_i(x, y)$ does not by itself compute that answer: with the nodal basis introduced on p. 11, $f(x_i, y_i, t) = \lambda_i(t)$ merely reproduces the prescribed coefficient.

In particular, *every* binary sequence inserted into (1) gives a nonincreasing positive sequence, since

$$\lambda_i \ge g_{i+1} \ge \lambda_{i+1}.$$

So this monotonicity does not select the correct SAT labels from incorrect ones. This observation is not a lower bound for SAT: a suitable input-dependent construction could add the missing information. The gap is that the manuscript does not prove that system $\mathcal{U}$, including its BBGKY evolution, can be initialized from the formulas without knowing the answers and will produce precisely their true labels [1, pp. 15–16]. Changing to the substitute basis mentioned on p. 17 does not supply that missing correctness argument.

3 An exact failure of the power-mean identity

Suppress the time variable and write w for the manuscript's stabilization function. For $N > 4$, put

$$A_r = \frac{1}{4}\sum_{i=1}^{4} \lambda_i^r, \qquad B_r = \frac{1}{N-4}\sum_{i=5}^{N} \lambda_i^r.$$

The relation preceding Claim 5.3 defines w by

$$A_1 w = B_1. \tag{2}$$

Corollary 5.4 then asserts, for the *same* w,

$$A_r w^r = B_r, \qquad 0 < r < 1, \tag{3}$$

and uses this as equation (2) of $\mathcal{U}$ [1, p. 15]. But (2) does not imply (3), even for positive, strictly decreasing coefficients.

Counterexample. Take $N = 8$, $r = \frac{1}{2}$, and

$$(\lambda_1, \dots, \lambda_8) = (64, 49, 36, 25, 16, 9, 4, 1).$$

Then

$$A_1 = \frac{87}{2}, \quad B_1 = \frac{15}{2}, \quad w = \frac{5}{29}, \quad A_{1/2} = \frac{13}{2}, \quad B_{1/2} = \frac{5}{2}.$$

Both sides of the proposed identity are positive, but their squares differ:

$$\left(A_{1/2}\sqrt{w}\right)^2 - B_{1/2}^2 = \frac{169}{4}\frac{5}{29} - \frac{25}{4} = \frac{30}{29} \neq 0. \tag{4}$$

Thus the mismatch is exact, not a rounding artifact. Appending $\lambda_{8+k} = 2^{-k}$ for $k \geq 1$ supplies a positive decreasing tail converging to zero and satisfies the stated summability requirement for $r = \frac{1}{2}$. A common positive rescaling also preserves the failure.

The missing condition can be written explicitly: because $w = B_1/A_1$, (3) is equivalent to

$$\frac{A_r}{A_1^r} = \frac{B_r}{B_1^r}.$$

Equality of these normalized power averages is an additional restriction, not a consequence of matching the ordinary means by a scalar.

Scope of the counterexample. It refutes this general algebraic passage under positivity, monotonicity and summability; it is not claimed to be a countermodel satisfying every other equation of $\mathcal{U}$. If special SAT-dependent constraints are intended to enforce (3), they must be stated, proved and shown compatible with representing arbitrary inputs without presupposing their answers. The omitted proof of Corollary 5.4 does not establish this. Replacing w by a new r-dependent scalar would change the system, not justify the displayed identity for its original w.

4 Approximation must preserve the decision bit

The manuscript treats g_j and g_{j+1} in the remaining range as indistinguishable [1, p. 16, paragraph before the complexity accounting]. Yet these endpoints encode opposite answers. Let $\Delta_i = g_i - g_{i+1} > 0$. For midpoint decoding, a sufficient uniform certificate is

$$|\widehat{\lambda}_i - \lambda_i| < \frac{\Delta_i}{2}. \tag{5}$$

With an allowed error of $\Delta_i/2$, the midpoint itself is consistent with either endpoint. Merely making a field contribution small therefore does not justify discarding it when its endpoint distinguishes yes from no.

A shrinking gap does *not* by itself prove superpolynomial cost: a gap $2^{-p(n)}$ can be resolved using only polynomially many fractional bits. What is missing here is a proved decision-preserving error bound, together with a polynomial-time method for attaining it on every input. Nor is a theorem supplied that the representative/permutation procedure preserves the labels of all unexamined instances [1, p. 16, item 2c].

5 The polynomial bound remains a proof obligation

The note $s^* = P(n)$ on p. 14 and the running-time labels in items 2a–2c on p. 16 are assertions, not a derivation of a uniform bound for the proposed computation [1, pp. 14, 16]. Section 5.1 describes numbering instances and retaining a polynomially bounded collection of values [1, p. 17]; this is not yet a procedure for computing their SAT labels.

A complete argument must relate the formula's encoded length n to the retained representation; construct its data; justify any truncation and convergence of the evolution; bound the cost and precision of its arithmetic; and prove both decoding and the treatment of unrepresented instances. Writing a differential or integral system, or calling its values computable, does not establish those costs. Nothing here proves that a polynomial implementation is impossible; the claim is that the manuscript does not establish one.

6 Conclusion and reproducibility

Theorem 5.1 is not proved as written. The power-mean identity is not valid under the stated general conditions, and even repairing it would leave the semantic-correctness, exact-decoding and complexity arguments unproved. These are objections to this proof, not a proof of $\mathsf{P} \neq \mathsf{NP}$.

The accompanying `verify.py` checks (4) with rational arithmetic, checks all 256 endpoint labelings for an eight-instance example, and tests the midpoint ambiguity and certified decoding condition. These checks corroborate the local calculations; they are neither a SAT solver nor a verification of the entire manuscript. The repository's unfinished claims ledger and full-review verdict are not completed by this note.

References

[1] Dmytro Topchyi. *The theory of plafales: P vs NP problem solution.* Manuscript dated 11 September 2026, 20 pages. Reviewed copy: `paper/P_vs_NP_problem_by_Topchyi_sep_2026.pdf` in `tarstars/pnp`. All manuscript page references refer to this version.

[2] Stephen Cook. *The P versus NP problem.* Clay Mathematics Institute, official problem description. `https://www.claymath.org/wp-content/uploads/2022/06/pvsnp.pdf`. Accessed 15 October 2026. Used only for the standard complexity definitions.